01. Using the **Barometer Method**, analyze each of the following algorithms. For each algorithm:

(a)
n = 5, 10 and 20

```
SumArray (A, n)
{
    sum = 0;
    for (i = 0; i < n; i++)
        sum = sum + A[i];
    return sum;
}
```

(b)
n = 3, 5 and 10

```
PrintPairs (A, n)
{
    count = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            count = count + 1;
}
```

(c)
n = 4, 8 and 16.

```
Multiply (A, n)
{
    result = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < i; j++)
            result = result + A[i] * A[j];
}
```

1. Identify the **basic (barometer) operation**.
2. Determine the **number of times** the basic operation is executed as a function of n.
3. Derive the **asymptotic time complexity**.
4. Calculate the number of executions of the basic operation for the given values of n.

02. **True/False**

A. If the growth function for an algorithm is expressed as polynomial terms, then the asymptotic complexity of the algorithm is determined by the term with the smallest exponent of the variable.

B. The asymptotic complexity, time complexity and order of an algorithm are the same concept.

C. All of the terms in a growth function (polynomial expression) contribute to the order of the function.

D. Time complexity is more important than space complexity when we design algorithms.

03.

a. A sorting algorithm is in **$O(n^2)$** in the worst case. It takes **50 seconds** to sort **10,000 records**, i.e., **n = 10,000**.

I. What is the predicted time for the algorithm to sort **20,000 records**?

II. Suppose that you are presented with a machine that is **49 times as fast**. How many records will you be able to process on the new machine in **5 seconds**?

Note: You are required to show all your workings.

b. Calculate the complexity of the following code fragment:

```
count1 = count2 = 0
for (i = 1; i <= n; i *= 2)
   for (j = 1; j <= i; j++)
      count1 = count1 + 1

for (i = 1; i <= n; i++)
   for (j = 1; j <= n; j++)
      count2 = count2 + 2
```

Note: You are required to show all your workings.

04. Assume the following functions describe the runtime of some algorithm. Determine **Big O** to simplify the algorithm's runtime complexity.

A. $f(n) = 3n^2 + 2n^3$

B. $f(n) = 20n^2 + 3246553343$

C. $f(n) = 25n^2 + 2 \cdot 2^n + 90 \log n$

D. $f(n) = 55 \log n + 102 \log n + 55 \log n + 2 \log n$

E. $f(n) = 36n \log n + 4n$

05. Determine the **time complexity**, in terms of n, of each of the following code fragments from 01 - 09

Code1

```
for (int i = 1 to n) {
  for (int j = i to n) {
     for (int k = j to n) {
        Sum += a[i]*b[j]*c[k]
     }
     if (gcd(i,j) == 1) {
        j = n
     }
  }
}
```

Code2

```
j = 1
while (j < n) {
   k = n/2
   while (k < n) {
      Sum += a[k]*b[k]
      k = k * k
   }
   j++;
}
```

Code3

```
j = 1
while (j < n) {
   k = j
   while (k < n) {
      Sum += a[k]*b[k]
      k++;
   }
   j += 0.25 * n
}
```

Code4

```
for (int j = 1 to n) {
  for (int k = j to n) {
     x = k
     while (x < n) {
        Sum += a[j]*b[k]*c[x]
        x = n + 1
     }
     if (x % 3 == 0) {
        x = n + 1
     }
  }  x = x + 1
}
```

Code5

```
j = 2
while (j < log n) {
   k = 3
   while (k < n) {
      Sum += a[k]*b[k]
      k = k * sqrt(k)
   }
   j = 2 * j
}
```

Code6

```
j = 2
while (j < n) {
   k = j
   while (k < n) {
      Sum += a[k]*b[k]
      k = k * k
   }
   j = sqrt(2) * j
}
```

Code7

```
for (int i = 1 to n) {
   j = i
   while (j < n) {
      k = j
      while (k < n) {
         Sum += a[j]*b[k]*c[x]
         k += log log n
      }
      j += log (j + 5)
   }
}
```

Code8

```
for (int i = 1 to n) {
   for (int j = i to n) {
      for (int k = j*j to n) {
         Sum += a[i]*b[j]*c[k]
      }
   }
}
```

Code9

```
int i =5
while (j < log n){
   int k =5
   while (k < n) {
      Sum += a[j] * b[k]
      k = k ^ 3
   }
   j = 1.3 * j
}
```